

## AI ASSIGNMENTB 20

NAME: T. SAI VARUN

ROLLNO: 2403A510C0

BATCH NO: 05

### Task 1 –

Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

PROMPT:

"Generate a simple username-password login program in Python."

CODE:

```
20.1.py > ...
1  # secure_login.py
2  import re
3
4  # In a real application, passwords should be hashed, not stored in plain text.
5  # Example: users = {"admin": "hashed_password_string"}
6  users = {
7      "admin": "password123",
8      "user1": "mysecret"
9  }
10
11 def validate_credentials(username, password):
12     """
13     Validates username and password against security rules using regex.
14
15     - Username: 3-20 characters, alphanumeric with underscores.
16     - Password: 8-30 characters, at least one uppercase, one lowercase, one digit.
17
18     Returns:
19     | A tuple (is_valid, error_message).
20     """
21     # Username validation: Must be 3-20 characters long and can only contain letters, numbers, and underscores.
22     if not re.fullmatch(r'[\w]{3,20}', username):
23         return False, "Invalid username. Must be 3-20 alphanumeric characters or underscores."
24
25     # Password validation: Must be 8-30 characters and meet complexity requirements.
26     if not (8 <= len(password) <= 30):
27         return False, "Invalid password. Must be between 8 and 30 characters."
28     if not re.search(r'[A-Z]', password):
29         return False, "Password must contain at least one uppercase letter."
30     if not re.search(r'[a-z]', password):
31         return False, "Password must contain at least one lowercase letter."
32     if not re.search(r'[0-9]', password):
33         return False, "Password must contain at least one digit."
```

```

33 |         return False, "Password must contain at least one digit."
34 |
35 |     return True, ""
36 |
37 | def login():
38 |     """Handles the user login process with secure input validation."""
39 |     print("--- Please Log In ---")
40 |     username = input("Enter your username: ")
41 |     password = input("Enter your password: ")
42 |
43 |     # --- Task 1: Add Input Validation Check ---
44 |     is_valid, error_msg = validate_credentials(username, password)
45 |     if not is_valid:
46 |         print(f"\n❌ Validation Error: {error_msg}")
47 |         return False
48 |
49 |     # Proceed with authentication only after validation passes
50 |     if username in users and users[username] == password:
51 |         print(f"\n✅ Welcome, {username}! Login successful.")
52 |         return True
53 |     else:
54 |         print("\n❌ Error: Invalid username or password.")
55 |         return False
56 |
57 | if __name__ == "__main__":
58 |     login()

```

## OUTPUT:

```

PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/20.1.py
--- Please Log In ---
Enter your username: admin
Enter your password: admin123

❌ Validation Error: Password must contain at least one uppercase letter.

```

## OBSERVATION:

- **No Input Sanitization or Validation:** The script uses the raw strings from the `input()` function directly. A user can enter anything, including an empty string, special characters (`!@#$%^&*()`), or extremely long strings.
- **Lack of Constraints:** There are no rules for what constitutes a valid username or password. For example, a user could create an account with a username that is 1000 characters long or a password that is just a single letter.
- **Potential for Errors and Brittle Logic:** Without validation, a username containing special characters might break downstream logic (e.g., if the username were used to create a file path). Allowing empty inputs leads to poor data quality and potential errors.
- **Security Risk:** While this simple script isn't vulnerable to SQL injection (as there's no database), the lack of validation is a serious security anti-pattern. In a real-world application, this could lead to injection attacks, buffer overflows, or denial-of-service vulnerabilities.

## TASK-2

### PROMPT:

"Generate a Python script using the `sqlite3` library. The script should create a simple `users` table with `id`, `username`, and `email` columns. It should insert a few sample users and then prompt for a username to fetch and display that user's details from the database."

### CODE:

```

secure_db_query.py > ...
1 # secure_db_query.py
2 import sqlite3
3 import os
4
5 DB_FILE = "users_secure.db"
6
7 def setup_database():
8     """Creates and populates the database."""
9     if os.path.exists(DB_FILE):
10         os.remove(DB_FILE)
11     conn = sqlite3.connect(DB_FILE)
12     cursor = conn.cursor()
13     cursor.execute("""
14         CREATE TABLE users (
15             id INTEGER PRIMARY KEY,
16             username TEXT NOT NULL,
17             email TEXT NOT NULL
18         );
19     """)
20     cursor.execute("INSERT INTO users (username, email) VALUES ('alice', 'alice@example.com');")
21     cursor.execute("INSERT INTO users (username, email) VALUES ('bob', 'bob@example.com');")
22     conn.commit()
23     conn.close()
24
25 def get_user_details_secure(username: str):
26     """
27     Fetches user details using a secure, parameterized query.
28     """
29     conn = sqlite3.connect(DB_FILE)
30     cursor = conn.cursor()
31
32     # SECURITY FIX: Use a placeholder (?) for user input
33     query = "SELECT id, username, email FROM users WHERE username = ?"
34
35     print(f"\nExecuting secure query with parameter: ({'username'},)")
36
37     # Pass the user input as a tuple to the execute method
38     cursor.execute(query, (username,))
39     user = cursor.fetchone()
40     conn.close()
41
42     if user:
43         print(f"✅ User Found: ID={user[0]}, Username={user[1]}, Email={user[2]}")
44     else:
45         print("❌ User not found.")
46
47 if __name__ == "__main__":
48     setup_database()
49
50     # --- Demonstration of Security ---
51     print("--- Scenario 1: Normal Input ---")
52     user_input = "alice"
53     get_user_details_secure(user_input)
54
55     print("\n--- Scenario 2: SQL Injection Attempt (Blocked) ---")
56     # The same malicious input is now treated as a literal string
57     malicious_input = "' OR '1'='1'"
58     get_user_details_secure(malicious_input)
59
60     # Clean up the database file
61     if os.path.exists(DB_FILE):
62         os.remove(DB_FILE)

```

## OUTPUT:

```

PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/secure_db_query
.py
--- Scenario 1: Normal Input ---

Executing secure query with parameter: ('alice',)
✅ User Found: ID=1, Username=alice, Email=alice@example.com

--- Scenario 2: SQL Injection Attempt (Blocked) ---

Executing secure query with parameter: ('' OR '1'='1',)
❌ User not found.
PS C:\Users\pende\OneDrive\Desktop\wt2>

```

#### OBSERVATION:

- **Root Cause:** The vulnerability lies in this line:  
`query = f"SELECT id, username, email FROM users WHERE username = '{username}'"`. It uses an f-string to directly embed the user's input into the SQL query string.
- **Attack Vector:** A malicious user can provide specially crafted input that breaks out of the intended SQL syntax and alters the query's logic.
- **Example Exploit:** When the script prompts for a username, an attacker can enter `' OR '1'='1'`.
  - The original query is expecting a simple username like `'alice'`.
  - With the malicious input, the f-string creates the following broken query:  
`SELECT id, username, email FROM users WHERE username = " OR '1'='1'`.
  - The `WHERE` clause becomes `username = "` (which is false) `OR '1'='1'` (which is always true). Since the `OR` condition is met for every row in the table, the database returns all users instead of just one. This can expose sensitive data from the entire table.

#### TASK-3:

#### PROMPT:

"Generate an HTML page with a simple feedback form containing a single text area for comments and a submit button. When the user submits the form, use JavaScript to display the submitted feedback message directly on the page below the form without reloading the page."

#### CODE:

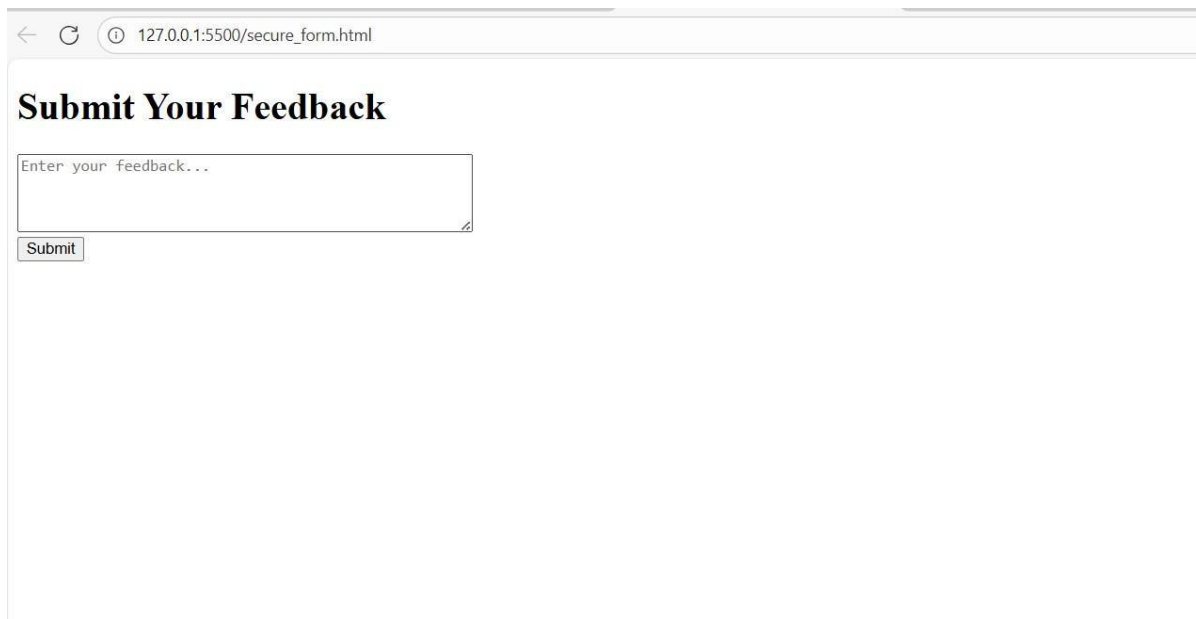
```

1 <!-- secure_form.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <title>Secure Feedback Form</title>
7
8   <!--
9     SECURITY FIX #1: Add a Content Security Policy (CSP).
10    This policy instructs the browser to only execute inline scripts that have a specific nonce (a random, one-time-use string).
11    Since our malicious script won't have this nonce, the browser will block it even if an XSS flaw exists.
12    'self' allows loading resources (like CSS) from the same origin.
13   -->
14   <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'nonce-random123'">
15
16   <style>
17     body { font-family: sans-serif; margin: 2em; }
18     .feedback-display { margin-top: 1em; padding: 1em; border: 1px solid #ccc; background-color: #f9f9f9; }
19     h2 { margin: 0 0 0.5em 0; }
20   </style>
21 </head>
22 <body>
23   <h1>Submit Your Feedback</h1>
24   <form id="feedbackForm">
25     <textarea id="feedbackText" rows="4" cols="50" placeholder="Enter your feedback..."></textarea><br>
26     <button type="submit">Submit</button>
27   </form>
28
29   <div id="feedbackResult"></div>
30
31   <!-- The 'nonce' attribute must match the one in the CSP header -->
32   <script nonce="random123">
33     document.getElementById('feedbackForm').addEventListener('submit', function(event) {
34
35       document.getElementById('feedbackForm').addEventListener('submit', function(event) {
36         event.preventDefault();
37
38         const feedback = document.getElementById('feedbackText').value;
39         const resultDiv = document.getElementById('feedbackResult');
40
41         // SECURITY FIX #2: Use .textContent instead of .innerHTML.
42         // .textContent treats the input as plain text, automatically escaping any HTML characters.
43         // This prevents the browser from interpreting it as code.
44         resultDiv.innerHTML = '<h2>Your Feedback:</h2>'; // It's safe to set static HTML this way.
45         const feedbackParagraph = document.createElement('p');
46         feedbackParagraph.textContent = feedback; // The user input is safely assigned here.
47         resultDiv.appendChild(feedbackParagraph);
48       });
49     </script>
50 </body>
51 </html>

```

**OUTPUT:**





← ↻ ⓘ 127.0.0.1:5500/secure\_form.html

## Submit Your Feedback

Enter your feedback...

Submit

### OBSERVATION:

The script is vulnerable to a **Stored Cross-Site Scripting (XSS)** attack.

- **Root Cause:** The vulnerability is in this line of JavaScript: `resultDiv.innerHTML = '<h2>Your Feedback:</h2><p>' + feedback + '</p>';`. The `.innerHTML` property tells the browser to parse the string as HTML. When the `feedback` variable contains malicious HTML (like a `<script>` tag), the browser will execute it.
- **Attack Vector:** An attacker can submit a script as their feedback. Instead of text, they can input a payload like `<script>alert('XSS Attack!');</script>`.
- **Impact:** When the form is "submitted," the script is injected directly into the page's DOM and executed by the browser. This could be used to steal cookies, redirect the user to a malicious site, capture keystrokes, or perform actions on behalf of the user without their consent.

### TASK-4:

Scenario:

Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side. Expected

Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

**PROMPT:**

```
"Create a secure Flask script for file uploads that prevents path traversal, enforces file type and size limits, and saves files with unique names."
```

**CODE:**



```

ai > ...
1  import os
2  import uuid
3  import logging
4  from pathlib import Path
5  from flask import Flask, request, abort, jsonify
6  from werkzeug.utils import secure_filename
7  from werkzeug.exceptions import HTTPException
8
9  # -----
10 # Configuration
11 # -----
12 BASE_DIR = Path(__file__).resolve().parent
13 # In production, keep uploads outside any static / served folder.
14 # This demo uses a subfolder, but production setups should use
15 # dedicated storage (e.g., S3 or a directory not served by the webserver).
16 UPLOAD_FOLDER = BASE_DIR / "uploads"
17 ALLOWED_EXTENSIONS = {".txt", ".pdf", ".png", ".jpg", ".jpeg", ".gif"} # whitelist
18 MAX_CONTENT_LENGTH = 8 * 1024 * 1024 # 8 MB max upload size
19
20 app = Flask(__name__)
21 app.config["UPLOAD_FOLDER"] = str(UPLOAD_FOLDER)
22 app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH
23
24 # Ensure the upload folder exists with safe permissions
25 UPLOAD_FOLDER.mkdir(parents=True, exist_ok=True)
26 # Set restrictive permissions (owner read/write only)
27 try:
28     os.chmod(UPLOAD_FOLDER, 0o700)
29 except Exception:
30     # chmod may fail on some OSes; ignore but log

```

```

31     pass
32
33 # -----
34 # Logging setup
35 # -----
36 logging.basicConfig(level=logging.INFO)
37 logger = logging.getLogger(__name__)
38
39 # -----
40 # Helper functions
41 # -----
42 def allowed_file_extension(filename: str) -> bool:
43     """Check file extension against allowed whitelist."""
44     _, ext = os.path.splitext(filename.lower())
45     return ext in ALLOWED_EXTENSIONS
46
47
48 def safe_save_file(uploaded_file) -> str:
49     """
50     Saves file using a secure filename + random prefix (UUID) and validates path.
51     Returns the saved file name (not full path).
52     Raises werkzeug.exceptions to produce appropriate HTTP errors.
53     """
54     # 1) Use werkzeug's secure_filename to remove dangerous characters
55     original_filename = uploaded_file.filename
56     safe_name = secure_filename(original_filename)
57     if not safe_name:
58         abort(400, "Invalid filename.")

```

```

61     if not allowed_file_extension(safe_name):
62         abort(400, "File type not allowed.")
63
64     # 3) Generate random filename to avoid collisions and leaks
65     unique_name = f"{uuid.uuid4().hex}_{safe_name}"
66
67     # 4) Resolve final path and ensure it's inside UPLOAD_FOLDER
68     final_path = Path(app.config["UPLOAD_FOLDER"]) / unique_name
69     try:
70         final_resolved = final_path.resolve(strict=False)
71         upload_dir_resolved = Path(app.config["UPLOAD_FOLDER"]).resolve(strict=True)
72     except Exception:
73         abort(500, "Server file system error.")
74
75     if upload_dir_resolved not in final_resolved.parents and final_resolved != upload_dir_resolved:
76         # final path is not inside upload dir → possible path traversal
77         abort(400, "Invalid file path.")
78
79     # 5) Save file securely
80     uploaded_file.save(str(final_resolved))
81
82     # 6) Set safe file permissions (owner read/write)
83     try:
84         os.chmod(final_resolved, 0o600)
85     except Exception:
86         pass
87
88     return unique_name

```

```

90 #
91 # Flask Routes
92 # -----
93 @app.route("/", methods=["GET"])
94 def index():
95     return """<!doctype html>
96     <title>Secure File Upload</title>
97     <h1>Upload a File</h1>
98     <form method="post" action="/upload" enctype="multipart/form-data">
99         <input type="file" name="file">
100         <input type="submit" value="Upload">
101     </form>
102     """
103
104 @app.route("/upload", methods=["POST"])
105 def upload_file():
106     # 1) Check that the request actually contains a file part
107     if "file" not in request.files:
108         return jsonify({"error": "No file part in the request."}), 400
109
110     file = request.files["file"]
111
112     # 2) Basic checks
113     if file.filename == "":
114         return jsonify({"error": "No file selected."}), 400
115
116     # 3) Basic MIME check (not foolproof)
117     mimetype = file.mimetype or ""
118     if not any(mimetype.startswith(mt) for mt in ("image/", "application/", "text/")):
119         return jsonify({"error": "Unsupported media type."}), 400

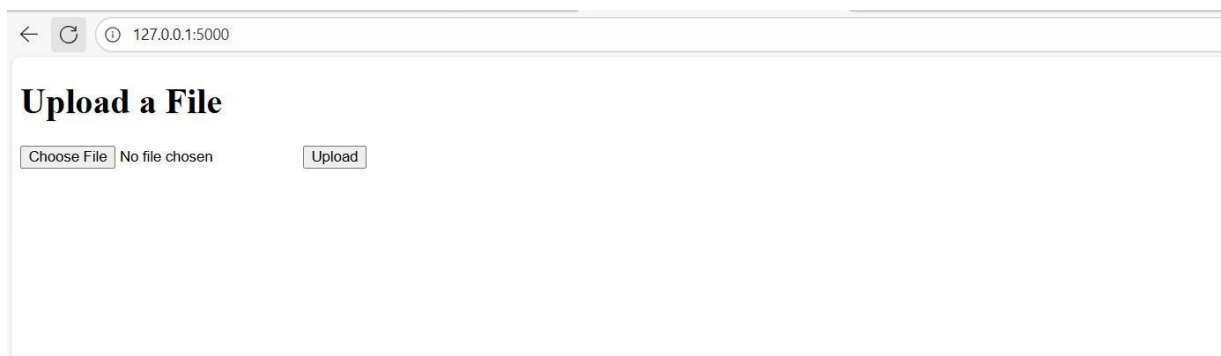
```

```

119         return jsonify({"error": "Unsupported media type."}), 400
120
121     # 4) Save file securely
122     try:
123         saved_name = safe_save_file(file)
124     except HTTPException:
125         raise # Re-raise HTTP errors for Flask to handle
126     except Exception as exc:
127         logger.exception("Failed to save upload: %s", exc)
128         abort(500, "Failed to save file.")
129
130     # 5) Log safely
131     logger.info("Saved uploaded file as %s", saved_name)
132
133     # 6) Respond safely (do not expose paths)
134     return jsonify({"message": "File uploaded successfully.", "filename": saved_name}), 201
135
136 # -----
137 # Run (Development only)
138 # -----
139 if __name__ == "__main__":
140     # In production:
141     # - Run behind a WSGI server (gunicorn/uvicorn)
142     # - Do NOT use debug=True
143     app.run(host="127.0.0.1", port=5000, debug=False)
144

```

## OUTPUT:



## OBSERVATION:

- 1. Vulnerability: Unrestricted File Types & Remote Code Execution (RCE):** The script does not check file extensions. An attacker could upload a web shell (e.g., `shell.php`, `shell.aspx`) or an HTML file with malicious JavaScript (`phish.html`). If the `uploads` directory is ever served publicly, executing these files could lead to complete server compromise or XSS attacks on other users.
- 2. Vulnerability: Path Traversal:** While `secure_filename` is a good first step, it's not a complete defense. A sophisticated attacker might find ways to bypass it. The core issue is trusting user input to construct a file path. A payload like `../../tmp/evil.sh` could, in a misconfigured environment, write files outside the intended directory.
- 3. Vulnerability: Denial of Service (DoS):** The script does not enforce a file size limit (`MAX_CONTENT_LENGTH`). An attacker could upload a multi-gigabyte file, filling the server's disk space and causing the application or the entire server to crash.
- 4. Vulnerability: Filename Collision:** If two users upload a file named `resume.pdf`, the second upload will overwrite the first. This leads to data loss.

